

CalTrack — Shared Database Setup Guide

This guide explains how two or more developers can connect to and share the same database and Redis containers for the CalTrack project.

Architecture Overview

Instead of running separate local database containers, you run a single database container on one host (either one developer's machine or a remote VPS) and connect both developers' Django applications to it:

```
`+-----+ +-----+
```

(Django App)

(Django App)

```
+-----+ +-----+
```

Connects to DB_HOST:5432

```
+-----+ +-----+
```

```
V V +-----+
```

(Docker Container)

```
+-----+`
```

Option 1: Shared Database on the Local Network (LAN)

If both developers are working in the same office or connected to the same Wi-Fi router, you can run the database on **Developer A's** computer.

Step 1: Find Developer A's Local IP Address

On Developer A's machine: 1. Open PowerShell and run: `ipconfig` 2. Look for the **IPv4 Address** of your active network connection (e.g., `192.168.1.100`).

Step 2: Run the Docker Container on Developer A's Machine

Start the services in background mode: `bash docker compose up -d` Note: The `docker-compose.yml` is configured with `ports: - "5432:5432"`, meaning it is accessible to all devices on the local network.

Step 3: Configure the Firewall (Developer A's Machine)

Windows Defender Firewall blocks incoming traffic by default. Allow incoming traffic on ports **5432** (PostgreSQL) and **6379** (Redis): 1. Run PowerShell as **Administrator**. 2. Run the following commands: `powershell New-NetFirewallRule -DisplayName "CalTrack Shared DB" -Direction Inbound -Protocol TCP -LocalPort 5432 -Action Allow New-NetFirewallRule -DisplayName "CalTrack Shared Redis" -Direction Inbound -Protocol TCP -LocalPort 6379 -Action Allow`

Step 4: Update `.env` Configurations (Both Developers)

On both Developer A's and Developer B's machines, edit `backend/.env`:

```
`ini
```

Replace 'localhost' with Developer A's IPv4 address

DB_HOST=192.168.1.100 DB_PORT=5432

Configure Redis url for Celery/Channels (if sharing Redis too)

REDIS_URL=redis://192.168.1.100:6379/0`

Option 2: Shared Database on a Remote VPS (AWS, DigitalOcean, etc.)

If developers are working remotely (from home/different locations), host the database container on a cloud VPS.

Step 1: Setup Docker on the VPS

Install Docker and Docker Compose on a basic Linux VPS.

Step 2: Run the Database

Upload the project's [docker-compose.yml](#) to the VPS and run: `bash docker compose up -d`

Step 3: Secure the VPS Firewall

Ensure port 5432 (Postgres) and 6379 (Redis) are open on the VPS. For security, restrict access to the public IP addresses of both developers: ``bash`

Example using UFW (Ubuntu)

`sudo ufw allow from <Developer_A_Public_IP> to any port 5432 sudo ufw allow from <Developer_B_Public_IP> to any port 5432``

Step 4: Update `.env` (Both Developers)

Update [backend/.env](#): `ini DB_HOST=<VPS_PUBLIC_IP_ADDRESS> DB_PORT=5432`

■ Important Shared DB Guidelines

1. **Migrations:** When a developer creates a new migration, they must push the code and run `python manage.py migrate`. The other developer should pull the code and they are instantly in sync. 2. **Data Deletion:** Running `docker compose down -v` on the host machine will delete the shared volume data. Ensure the host container is kept running.